RAI: Recursive Accelerated Imitation for Robot Manipulation

Anonymous Author(s)
Affiliation
City, Country
email@domain

***Abstract*—Policies trained by behaviour cloning inherit the cautious tempo of human demonstrations and can fail in states absent from the training data. We introduce *Recursive Accelerated Imitation* (RAI), which closes the loop between imitation learning and deployment by training successive policies on their own accelerated, human-corrected execution. Each round executes the current policy above demonstration speed, records its closed-loop rollouts with human takeover, and trains the next policy on the accumulated episodes. This allows the policy to learn from states and contact conditions induced by its own closed-loop execution, while using human intervention only for targeted correction. With equal training episodes, RAI achieves 92.3% mean full-trial success across four contact-rich manipulation tasks, compared with 62.5% for the same architecture trained purely on teleoperation and 40.3% for $\pi_{0.5}$. The RAI dataset comprises (54.8%) human-controlled and (45.2%) autonomous frames. On a separate cyclic benchmark, RAI sustains roughly $2.46\times$ the pick rate of the round 1 teleoperation-only policy, showing that reliability and execution speed of the policy can improve together. Removing force conditioning reduces mean success to 77.3%, remaining above the teleoperation baseline. Together, these results support recursive collection and learning is effective to improve policy reliability while learning faster execution from policy-generated and human-corrected experience.**

***Index Terms*—Imitation learning, robot manipulation, behaviour cloning, contact-rich manipulation.**

I. Introduction

Imitation learning has enabled robots to acquire a wide range of manipulation skills from human demonstrations [1]–[3]. However, contact-rich manipulation remains challenging, because success depends on the interaction dynamics that emerge during execution. Teleoperation-only behaviour cloning is trained on state–action distributions generated by human demonstrations. While deployed, it is the first time the policy closes the control loop by its own actions – different data distribution. This mismatch creates two limitations: policy errors can drive the robot into states absent from the training data, and the policy inherits the slow and cautious execution tempo of human demonstrations [4]. Discrepancies between human-generated training data and policy-generated deployment data become crucial for contact-rich tasks.

Although fixed teleoperation datasets provide strong supervision, they do not directly capture the state and contact distributions produced by the deployed policy. This motivates us to make deployment part of data collection: execute the policy, correct failures, and train again on the resulting experience. This retains supervised imitation learning and allows successive policies to learn from state distributions via their own closed-loop experience. Moreover, for contact-rich manipulation, these observations should include contact forces alongside vision and proprioception, so that the policy can condition its actions on the real physical forces as well.

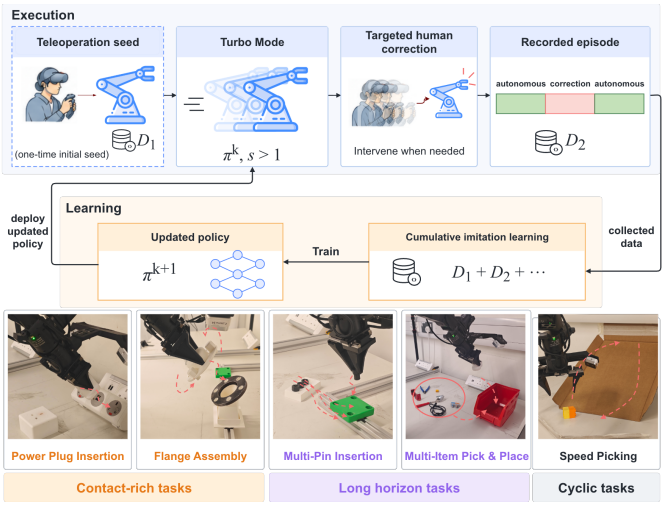


Fig. 1. Starting from teleoperation demonstrations, RAI collects accelerated rollouts under *Turbo mode* with targeted human corrections and trains successive policies on the cumulative dataset. *Bottom:* Five real-robot manipulation tasks evaluating across contact-rich, long-horizon, and speed-critical settings.

Further, real-world manipulation requires task speed and efficiency. Teleoperation is often deliberately cautious, and behaviour cloning inherits this slow tempo. This motivates collecting *accelerated policy rollouts* on the robot, with targeted human correction where needed, so that successive policies learn from accumulated real robot experience over time.

In this work, we introduce *Recursive Accelerated Imitation* (RAI), which makes policy execution itself part of data collection (Fig. 1). Rather than learning purely from human-generated demonstrations, each successive policy is trained on experience generated by its predecessor via closed-loop execution, supplemented by targeted human correction where needed. Therefore, RAI exposes the learner to the states and contact conditions generated by robot behaviour, while retaining human supervision for failures or undesirable actions.

Following the teleoperation demonstrations, RAI executes the current policy faster than the demonstration speed using real-time chunking [5] and playback scaling, records con-

tinuous episodes with human corrections when needed, and trains the next policy on the cumulative dataset. Apart from occational human corrections, the policy runs autonomously, so later rounds collect training data without full teleoperation.

Accelerated execution achieves two meaningful outcomes. First, faster motion becomes part of the new learned behaviour because the policy is trained on trajectories generated at higher speed. Second, faster execution changes the states and contact conditions encountered by the robot, creating training data that are not available from nominal-speed rollouts or teleoperation. Human corrections provide supervision over failures, and such resulting experience makes the next policy more robust.

Our contributions are:

- **Recursive Accelerated Imitation (RAI).** A deploy–correct–retrain loop in which each policy generates accelerated closed-loop data for training its successor, with targeted human corrections.
- **Accelerated execution for autonomous data collection.** RAI accumulates real-robot data from motions executed at higher speed, exposing subsequent policies to new resulting state and contact conditions.
- **Force-conditioned multimodal policy.** A 96.4 M-parameter flow-matching transformer capable of ingesting all available robot modalities jointly, including contact forces, demonstrating effectiveness in contact-rich manipulation.
- **Comprehensive real-robot evaluation.** Across contact-rich, long-horizon, and cyclic tasks, RAI demonstrates joint improvements in both the reliability and execution throughput, reaching 92.3% mean full-trial success and roughly $2.46\times$ the cyclic pick rate of the round 1 teleoperation-only policy.

With equal total training episode counts per task, the final RAI policy achieves 92.3% mean full-trial success, compared to 40.3% for $\pi_{0.5}$ (Table II). With fixed architecture and force conditioning, RAI improves the mean from 62.5% (pure teleoperation) to 92.3% (Table I), isolating the contribution of the data loop. Across the complete four-round RAI dataset, only 54.8% (Figure 7) of retained frames come from active human control; across the post-seed collection rounds, this reduces to 38.2% (Figure 7).

RAI improves both reliability and execution speed. On a separate cyclic task, the final policy performs about $2.46t\times$ the throughput of the round 1 teleoperation-only policy, showing that RAI can improve reliability and learned execution speed together. Contact-force conditioning further increases full-trial success from 77.3% to 92.3% (Table I).

II. Related Work

Large-scale behaviour cloning [6]–[8] and generative action heads [2], [3], [9] define the architecture we build on; the policy used in RAI is closest to π_0 [3], though trained from scratch on one platform, so we claim generalisation only within a task domain. Chunked prediction [1] creates disagreement at chunk boundaries, addressed by guided sampling [10], [11] or by real-time chunking [5], which freezes committed actions and inpaints the rest at inference so that computation overlaps execution. RAI uses the soft guidance of real-time chunking [5] at inference, guiding each new chunk toward the actions still to be executed. Cheaper sampling and faster runtimes [12]–[14] lower the cost of *deciding*, not how fast the robot then moves.

Prior works tackle either execution speed [4], [15]–[22], or reliability [23]–[27], but not both: acceleration methods do not learn from failures at higher speed, while correction methods do not train the policy to move faster. To our knowledge, no prior method improves both within a single collection loop.

Work on policy execution speed falls into three families. Test-time methods modulate playback while leaving the policy fixed: SAIL [15] reaches $3.2\times$ on hardware, while others vary the rate across phases of a task [16] or obtain it from temporal ensembling [18]; because the policy is unchanged, the fast rollout is discarded rather than learned from. Augmentation methods re-time demonstrations and retrain [4], [19]–[21] and report the strongest results so far [20]. Re-timing a demonstration only changes the playback speed, but its images, wrenches and torques were still recorded during the original slower motion, and resampling cannot produce observations of the contact dynamics resulted by faster motions. In contrast, executing the policy faster directly generates these observations, and corrective supervision turns the faster rollouts into successful real-world training data for the next policy. I2RLC [28] recovers references at three to ten times the original rate, but still from a fixed demonstration set. Reinforcement learning methods tune speed against a reward [17], [22], avoiding re-timed data but requiring reward design and on-robot exploration at speed. RAI records the deployed policy's accelerated rollouts, corrected by a human where it fails, and trains the next policy on them: supervision from states and contacts actually executed at speed, with no reward or value function.

Covariate shift between the standard behavior cloning training distribution and policy rollouts motivates collecting corrections on the policy's own state distribution [23], [24], scaled to continuous deployment [25] and extended to vision–language–action models and dexterous hands [29]–[31]. Others treat deployment as a training stage through self-generated data or real-world reinforcement learning [26], [32]–[34]. Almost all of this work optimises reliability and reports speed, if at all, as a side effect; the exception is $\pi^*_{0.6}$ [27], which reaches comparable compounding through advantage conditioning on a learned value function. RAI instead aggregates human-corrected accelerated rollouts into each round's training set, addressing covariate shift through supervised imitation alone.

Force and tactile conditioning improves contact-rich performance [35]–[38], though whether an explicit force channel adds information beyond kinematics remains contested [39]. Since force sensors are costly, external torque is often estimated from proprioception [40], [41] and used for force-feedback teleoperation [42], [43]. Our implementation follows FACTR 2 [44] for external-torque estimation. We measure a 15.0-point contribution of force conditioning to mean full-trial success under RAI (Table I).

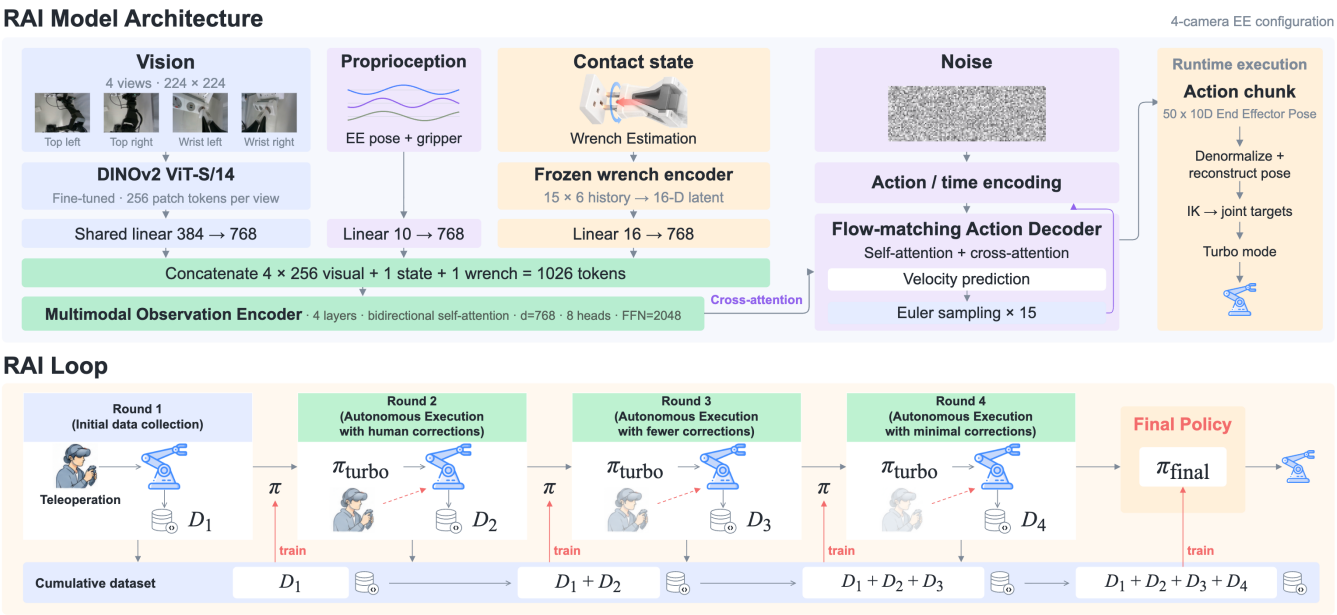


Fig. 2. RAI system. *Top:* RAI model architecture. Vision, proprioception and a compressed contact wrench are projected to a common width and concatenated into one context sequence, consumed once per chunk by the multimodal encoder backbone. The action denoiser cross-attends to that cached context while integrating the flow-matching ODE, and emits a 50-step chunk of end-effector pose deltas at 30 Hz. The contact wrench is estimated from joint signals with no force/torque sensor. *Bottom:* RAI collection and training loop. Round 1 trains on teleoperation alone; each later round deploys the previous policy at a requested scale of $s = 2$, records its execution with human corrections where needed, and trains the next policy on the cumulative dataset.

III. Methodology

RAI recursively turns policy deployment into training data, and repeats four steps: accelerate the current policy's execution, record rollouts with human corrections, aggregate the episodes, and train the next policy by imitation. We first describe this recursive collection and training loop (§III-A), then the accelerated execution and correction mechanisms used during collection (§III-B), and finally the multimodal flow-matching policy trained on the resulting data (§III-C).

A. Recursive Accelerated Imitation Loop

1) Recursive Collection and Cumulative Training: Let $\mathcal{D}_1$ denote the initial teleoperation dataset, and let $\pi^{(1)}$ be the policy trained on it. To collect round $k{+}1$, we deploy $\pi^{(k)}$ at a requested playback scale of $s = 2$. The operator intervenes proactively when the policy approaches a likely failure or when its behaviour requires correction, for example during an undesirable grasp approach. Rollouts are recorded continuously throughout autonomous execution, human correction, and subsequent policy execution.

Let $\mathcal{C}_{k+1}$ denote the episodes collected in round $k{+}1$. RAI updates the cumulative dataset and trains the next policy as

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \mathcal{C}_{k+1}, \qquad \pi^{(k+1)} = \mathrm{Train}_{\mathrm{IL}}(\mathcal{D}_{k+1}), \tag{1}$$

where $\mathrm{Train}_{\mathrm{IL}}$ uses the supervised flow-matching objective described in §III-C. Round 1 therefore contains only teleoperation data, while each subsequent round adds experience generated by the preceding policy. Human corrections stay within the same continuous episodes as autonomous policy executes. The complete collection and training loop is illustrated in Fig. 2.

B. Accelerated Execution and Human Correction

1) Real-Time Chunking and Turbo Playback: The policy predicts 50-step action chunks at 30 Hz, while the arm is commanded at 200 Hz by interpolating between consecutive policy actions. A new policy request is issued when the current chunk takes over. The request contains the current observation and the unexecuted suffix of the active chunk, re-expressed in the new end-effector reference frame.

Because inference is not instantaneous, the next chunk is scheduled to take over D policy actions after the observation is captured. Its first D predicted actions correspond to motion already underway by the time the chunk takes over and are therefore discarded. We set D from the 95th-percentile round-trip inference latency, converted to policy actions with a safety margin, and clamp it to $6 \le D \le 20$. Consequently, only the portion of each chunk before the next takeover is executed.

The new chunk is guided towards the unexecuted actions of the previous chunk during denoising [5]. Let $\tilde{A}$ denote these actions and $\hat{A} = x_t - t\,v_\theta$ the current estimate of the clean chunk. At each denoising step, the predicted velocity is updated as

$$v_\theta \leftarrow v_\theta - \min(\beta, c_t)\, W\big(\tilde{A} - \hat{A}\big), \tag{2}$$

where c_t is the time-dependent gain of [5], capped at $\beta = 10$. The diagonal weights W are 1 for the first D actions, decrease linearly to 0 at action 30, and are 0 thereafter. No action is explicitly overwritten; the remaining actions of the previous chunk instead guide generation of the new chunk.

Turbo mode scales the action playback interval as

$$\Delta t_{\mathrm{act}} = \frac{\Delta t_{\mathrm{pol}}}{s},$$

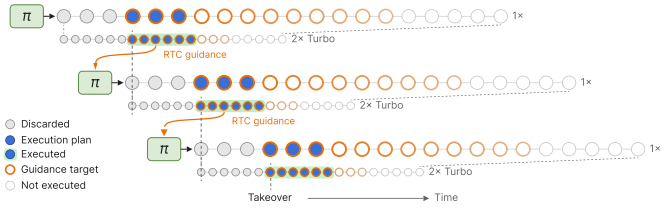


Fig. 3. Turbo playback with real-time chunking. The policy π predicts action chunks at the nominal policy rate, while turbo shortens the interval between executed actions. Actions that elapse while the next chunk is computed are discarded from its prefix, and the new chunk takes over at the dashed line. The unexecuted suffix of the previous chunk guides the next generation, with guidance weight decreasing along the chunk (ring opacity).

so the active policy trajectory is played at s times the nominal rate without changing its predicted waypoints. Under RTC, a new chunk can replace the active chunk before all 50 predicted actions are executed. Faster playback also causes more actions to elapse during each inference call, so a larger prefix of the next chunk is discarded before execution (Fig. 3).

2) Contact-Aware Teleoperation and Correction: Demonstrations and corrective interventions are collected through an XR headset driving a differential inverse-kinematics controller. The estimated contact wrench (§III-C) is rendered as real-time audio feedback, providing immediate cues for contact force magnitude. The operator uses this feedback to detect contact and limit excessive force during both teleoperation demonstrations and RAI corrections.

C. Multimodal Flow-Matching Policy

1) Multimodal Observations and Force Annotation: Each frame carries measured and commanded joint state, RGB images from four Intel RealSense D405 cameras. Two global cameras mounted above the workspace (top-left and top-right) and two wrist cameras (left & right). Each camera records 1280×720 RGB at 30 Hz, synchronised with the policy rate. We post-process the recorded joint signals to annotate the episodes with the force estimates described in Section §III-C.

2) Flow-Matching Architecture and Objective: The policy is a 96.4 M-parameter flow-matching transformer mapping a multimodal observation to a chunk of 50 future actions (Fig. 2). Each image observation passes through a fine-tuned DINOv2 ViT-S/14 backbone. Patch tokens are lifted to the model width $d = 768$ by a linear layer.

The model comprises of a four-layer multi-modal observation encoder that runs once per chunk with fully bidirectional self-attention and a six-layer action head that denoises the action chunk with full self-attention while cross-attending to that encoded context. Because the context is invariant across denoising steps, the action expert's cross-attention keys and values are computed once per chunk and reused, enabling high-frequency action sampling with a 15-step ODE solver.

Training follows the flow-matching objective: with interpolant $x_t = t\varepsilon + (1-t)a$ and target $u = \varepsilon - a$ the network predicts the straight-line velocity directly. Actions are 10-vectors of position, a six-dimensional rotation representation and a gripper command, expressed as *chunk-local* deltas against a reference latched at the start of the chunk [45]: $\Delta p = R_{\text{ref}}^{\top}(p_t - p_{\text{ref}})$ and $R_{\Delta} = R_{\text{ref}}^{\top} R_t$, with the gripper absolute. The reparameterisation is dataset-side, normalisation statistics are computed over the deltas.

3) Contact Force Estimation and Encoding: Our robotic arm has no force/torque sensor. Contact is inferred from joint currents (Fig. 5) by combining two external-torque estimators with complementary characteristics. A momentum observer estimates the residual between measured and model-predicted generalised momentum, providing reliable external torque but with a relatively high noise. A learned estimator [44] replaces the analytical rigid-body term with a network trained on contact-free motion, giving $\hat{\tau}_{\text{ext}} = \tau_m - f_\theta(x)$: a more accurate external torque estimation during free-space movements. The two are fused by a convex blend gated on motion speed, $\hat{\tau} = w\hat{\tau}_{\text{MO}} + (1-w)\hat{\tau}_{\text{NEXT}}$ with $w = \exp(-\ln 2\,(\bar{\dot{q}}/v_{1/2})^2)$, so the analytic observer supplies magnitude quasi-statically and the learned model supplies more accurate estimate during motion. Such fused torques are converted to an end-effector wrench through

$$\hat{F} = \left(J(q)^{\top}\right)^{\dagger} \hat{\tau}_{\text{ext}}$$

in a grasp frame. A pre-trained autoencoder compresses the wrench to a 16-dimensional token that is fed into the policy.

IV. Experiments

We evaluate whether RAI improves reliability over teleoperation at a matched episode budget, how reliability and sustained throughput evolve across recursive rounds, how recursive collection changes the training data, and the contribution of contact-force conditioning. Four manipulation tasks measure reliability, while a separate cyclic task measures sustained throughput.

A. Experimental Design

1) Tasks and Collection Protocol: The four reliability tasks are *power plug insertion*, *multi-pin insertion*, *multi-item pick-and-place*, and *flange assembly* (Fig. 4). Two are single-insertion tasks and two are long-horizon sequences. In both collection and evaluation, the pick location is randomised within a 297×420 mm workspace region.

The fifth task measures cyclic manipulation: the robot picks a block, drops it at the top of a slide, and picks it up again after it returns to a randomised location at the bottom. The task resets itself and therefore runs continuously without human intervention during evaluation.

For each reliability task, every round contributes 100 episodes, giving 400 episodes over four rounds. For the multi-item tasks, a collection episode contains a single item, such as one pin insertion or one pick-and-place transfer; evaluation instead tests the complete sequence. The cyclic task follows the same four-round protocol at half the budget, with 50 episodes per round and 200 in total. Round 1 is teleoperated. Each subsequent round deploys the preceding checkpoint at a requested playback scale of $s = 2$, records its execution with human corrections, and trains the next policy on the cumulative dataset (§III-A).

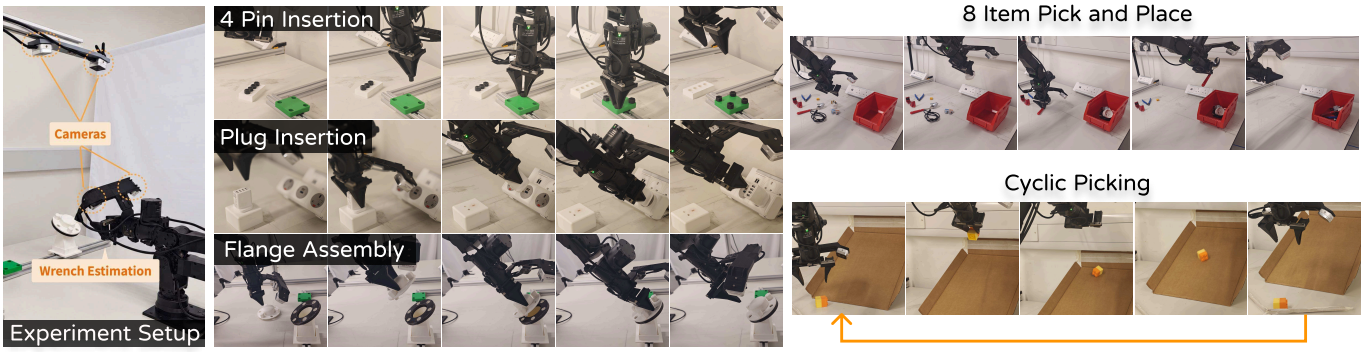


Fig. 4. Real robot experimental setup and representative execution sequences for four contact-rich manipulation tasks: power plug insertion, flange assembly, multi-pin insertion, multi-item pick-and-place, and one cyclic manipulation task: speed picking.

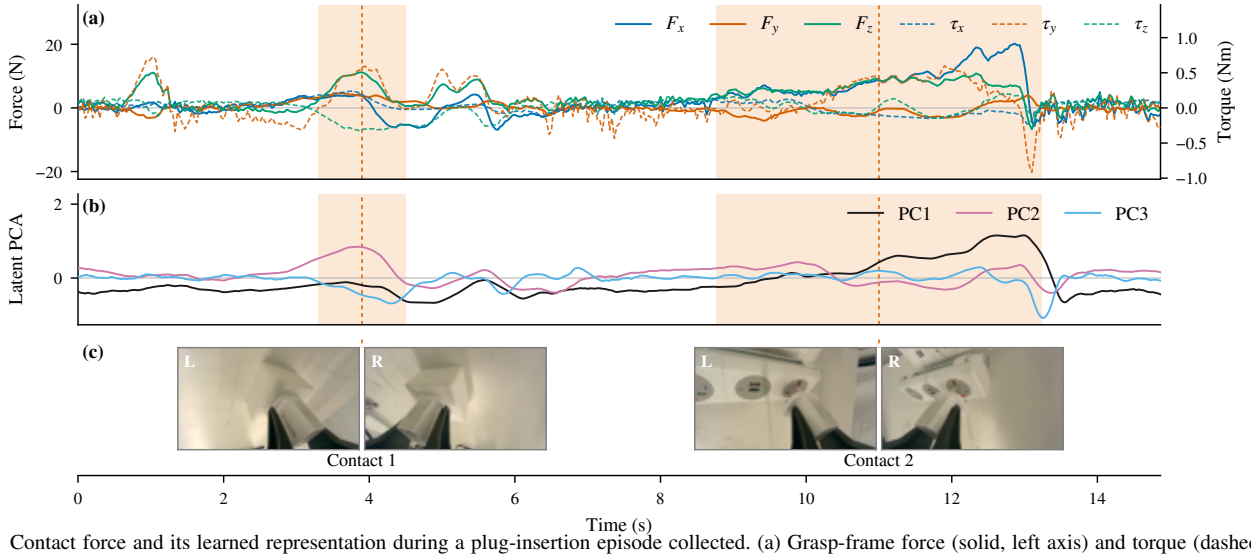


Fig. 5. Contact force and its learned representation during a plug-insertion episode collected. (a) Grasp-frame force (solid, left axis) and torque (dashed, right axis), estimated from joint signals without a force/torque sensor; both axes share the same zero. (b) The 16-dimensional contact-wrench latent consumed by the policy, projected onto its first three principal components, fitted over the full plug-insertion dataset and together explaining 67% of its variance. (c) Left (L) and right (R) wrist-camera views at the dashed instants. Shaded regions mark contact phases: grasping the plug (Contact 1) and inserting it into the socket (Contact 2). The latent distinguishes the two phases: PC2 responds to the grasp, while PC1 rises throughout insertion and drops sharply at release.

2) Controlled Comparisons: Table I provides the primary controlled comparison of the RAI collection strategy. Using the same policy architecture, we compare 400 episodes collected through pure teleoperation with the 400-episode RAI dataset, with force conditioning either enabled or disabled. Within each collection condition, the force-enabled and force-disabled models are trained on exactly the same episodes. Holding force conditioning fixed therefore compares collection strategies at a matched episode budget, while holding the collection strategy fixed measures the contribution of force observations. Episode counts are matched, but frame counts are not: the pure-teleoperation dataset contains more frames because episodes collected in later RAI rounds are shorter.

Table II compares the final RAI policy with $\pi_{0.5}$ as an external benchmark. $\pi_{0.5}$ is trained on the same matched-budget teleoperation dataset used in Table I, but receives one global and two wrist-camera views and no contact-wrench embedding, whereas RAI uses four camera views and force conditioning. This comparison therefore benchmarks overall task performance rather than isolating the effect of RAI collection.

The cyclic task evaluates progression across RAI rounds. Round 1 is trained on the 50-episode teleoperated seed, and each subsequent round adds 50 episodes to the cumulative dataset. This comparison therefore characterises performance as recursive experience accumulates; unlike Table I, it does not isolate collection strategy at a matched final episode budget.

3) Evaluation Metrics and Analysis:

a) Reliability and throughput: Each reliability task is time-limited and scored as success or failure. We evaluate 50 trials for power plug insertion and flange assembly, and 20 trials for multi-pin insertion and multi-item pick-and-place, with all policies evaluated at $s = 1$. Long-horizon trials are scored all-or-nothing and terminate at the first failure; remaining items are not attempted. Tables II and I additionally report per-item success as a diagnostic. Mean full-trial success is the unweighted mean across the four reliability tasks.

Trial duration is not directly comparable across these tasks because failures, timeouts, and recovery behaviour affect both completion time and which trials contribute to it. We therefore measure task efficiency separately on the cyclic task using

TABLE I

ABLATION OF COLLECTION METHOD AND FORCE CONDITIONING AT EQUAL EPISODE COUNTS. SUCCESS RATES (%); PIN AND PNP REPORT PER-ITEM AND FULL-SEQUENCE COMPLETION. MEAN AVERAGES FULL-TRIAL SUCCESS. BEST, INCLUDING TIES, IN BOLD.

Training	Force	Plug	Asm	Pin		PnP		Mean
				Item	All 4	Item	All 8	
Teleop	Off	74.0	66.0	15.0	0.0	92.5	55.0	48.8
	On	92.0	68.0	56.3	35.0	93.1	55.0	62.5
RAI (R4)	Off	86.0	68.0	73.8	65.0	**98.1**	**90.0**	77.3
	On	**98.0**	**96.0**	**92.5**	**85.0**	**98.1**	**90.0**	**92.3**

TABLE II

SUCCESS RATES (%). THE FINAL RAI AND $\pi_{0.5}$ POLICIES USE EQUAL TOTAL EPISODES PER TASK; EARLIER ROUNDS USE THE DATA ACCUMULATED SO FAR. PIN AND PNP REPORT PER-ITEM AND FULL-SEQUENCE COMPLETION. MEAN AVERAGES FULL-TRIAL SUCCESS. BEST IN BOLD.

Method	Plug	Asm	Pin		PnP		Mean
			Item	All 4	Item	All 8	
$\pi_{0.5}$	60.0	36.0	13.8	5.0	83.1	60.0	40.3
RAI (Recursive Accelerated Imitation)							
Round 1	34.0	22.0	10.0	0.0	53.0	10.0	16.5
Round 2	74.0	60.0	45.0	25.0	75.0	45.0	51.0
Round 3	88.0	84.0	85.0	60.0	90.1	75.0	76.8
Round 4	**98.0**	**96.0**	**92.5**	**85.0**	**98.1**	**90.0**	**92.3**

sustained throughput. Each policy is evaluated continuously at $s = 1$ for 10 minutes, and throughput is reported as completed picks per minute.

b) Human control during collection: Fig. 7 reports the fraction of retained frames under active human control in each newly collected round, excluding data from previous rounds. This is a frame-level measure of active intervention during recorded episodes rather than total operator time, which also includes setup and resets.

c) State and contact distributions: Fig. 8 visualises collected samples using t-SNE. Each sample comprises a 7-dimensional joint state and a 16-dimensional contact-wrench latent, with the two components standardised and weighted equally.

B. Results

1) Matched-Budget Benefit of Recursive Collection: Across RAI rounds, both dataset size and composition change. To evaluate the collection strategy at a fixed episode budget, Table I compares RAI with pure teleoperation using the same architecture under matched force-conditioning settings. With force conditioning, RAI increases mean full-trial success from 62.5% to 92.3%; without force conditioning, it increases success from 48.8% to 77.3%. Thus, at matched episode counts, RAI improves mean success by 29.8 and 28.5 percentage points, respectively. The benefit of recursive collection therefore persists with and without force conditioning.

2) Reliability Across Recursive Rounds: Mean full-trial success increases across successive RAI rounds, from 16.5% in Round 1 to 51.0%, 76.8%, and 92.3% in Rounds 2–4, respectively, with improvements across all four tasks. The final policy completes all four pin insertions in 85.0% of trials and all eight pick-and-place transfers in 90.0%.

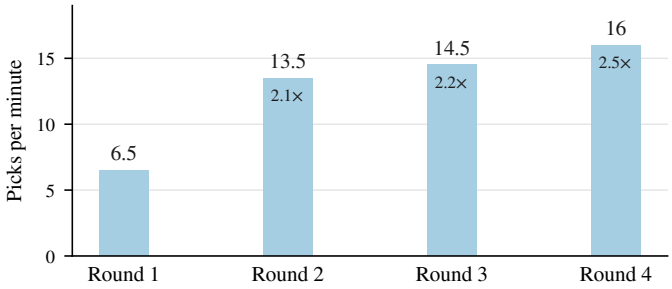


Fig. 6. Sustained throughput on the cyclic pick-and-place task across RAI rounds, measured as completed picks per minute over a continuous 10-minute window. Round 1 is trained on teleoperation alone; each later round adds rollouts collected with accelerated execution and human corrections. Values inside the bars give the throughput relative to Round 1.

As an external benchmark, the final RAI policy reaches 92.3% mean full-trial success, compared with 40.3% for $\pi_{0.5}$ (Table II), a difference of 52.0 percentage points. Because the models differ in architecture and observation modalities, this comparison benchmarks overall task performance rather than isolating the effect of RAI collection.

3) Sustained Throughput: Fig. 6 reports sustained throughput on the cyclic task. Round 1, trained on the teleoperated seed alone, completes 6.5 picks per minute. Throughput increases to 13.5, 14.5, and 16.0 picks per minute in Rounds 2–4, respectively, with the final policy achieving 2.46× the throughput of Round 1. Most of the observed increase occurs between Rounds 1 and 2, with smaller gains thereafter.

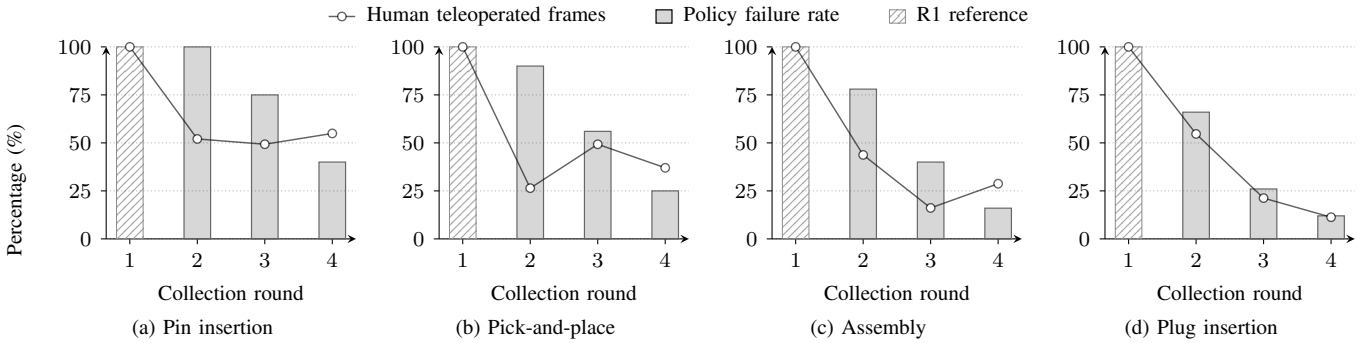
All policies are evaluated at $s = 1$, so the throughput increase is not produced by evaluation-time playback scaling. Together with the reliability results, this shows that task success and sustained throughput both improve across recursive rounds on their respective evaluation tasks.

4) Data Composition Under Recursive Collection:

a) Human control during collection: Across the complete four-round RAI dataset, 54.8% of retained frames are under active human control. Excluding the fully teleoperated seed round, this fraction falls to 38.2% across subsequent collections (Fig. 7). Later rounds therefore combine substantial autonomous execution with targeted human correction rather than requiring full teleoperation. The active-control fraction is below the Round 1 level for every task, although it does not decrease monotonically across all rounds.

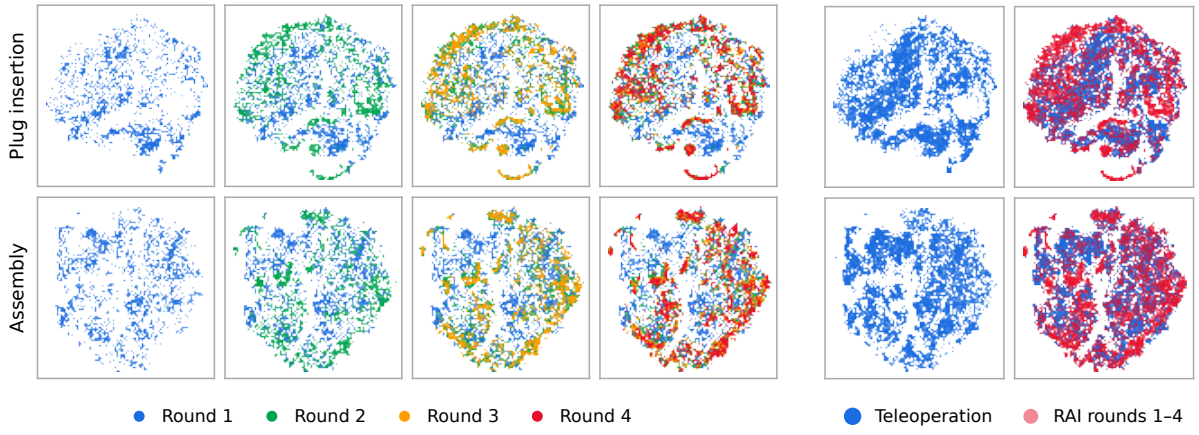
b) State and contact distributions: Fig. 8 examines how the collected state and contact representations change across rounds. In the t-SNE projections, later rounds populate regions sampled sparsely by the teleoperated seed. The accumulated RAI and matched-sample teleoperation datasets overlap broadly but also occupy different regions. The difference is therefore not simply a uniform expansion of coverage: the projected datasets have similar spread, while each contains regions sampled sparsely by the other.

The classifier analysis provides an additional quantitative measure of this distributional difference. Frames from Rounds 2–4 are distinguishable from teleoperation with AUCs



(a) Pin insertion (b) Pick-and-place (c) Assembly (d) Plug insertion

Fig. 7. Human control during data collection and failure rate of the collecting policy. Lines show the fraction of retained training frames under active human control, pooled across episodes newly collected in each round; earlier data are excluded. Translucent bars show $1 - \mathrm{SR}$ for the policy used to collect that round: collection rounds 2, 3, and 4 correspond to policies trained through rounds 1, 2, and 3, respectively. Round 1 is pure teleoperation; its dashed 100% bar is a reference. All panels share the same 0–100% scale.



(a) Data accumulated over RAI rounds, coloured by collection round. (b) Teleoperation, RAI Rounds 1-4 overlaid

Fig. 8. Distribution of collected samples in the joint-state and contact-wrench representation, projected to two dimensions with t-SNE. Each frame contains a 7-dimensional joint state and a 16-dimensional contact-wrench latent; the two blocks are standardised and weighted equally. Rows show plug insertion and assembly. (a) Data accumulate over RAI rounds. (b) Teleoperation is shown alone and with accumulated RAI data overlaid; each dataset contains 12 000 sampled observations. The overlays provide a qualitative comparison of the sampled distributions. See §IV-B4 for the corresponding analysis.

of 0.96–0.99 across the four tasks, compared with 0.69–0.78 for Round 1, which is itself teleoperation collected in separate sessions. Together, these analyses show that recursive collection changes which state and contact conditions are represented in the training data. Because accelerated execution and human correction occur together during collection, these experiments evaluate their combined effect and do not isolate acceleration alone.

5) Contribution of Force Conditioning: We evaluate contact-force conditioning within the 96.4 M-parameter multi-modal policy described in §III-C. With RAI collection fixed, enabling force raises mean full-trial success from 77.3% to 92.3%, a gain of 15.0 percentage points (Table I). The gains are task-dependent: assembly improves from 68.0% to 96.0%, multi-pin insertion from 65.0% to 85.0%, and power plug insertion from 86.0% to 98.0%, while multi-item pick-and-place remains at 90.0%.

In particular, assembly shows a strong combined effectiveness. Using teleoperation without force yields 66.0%, enabling force alone or switching to RAI alone reaches 68.0%; in contrast, combining RAI with force reaches 96.0%. These results show that force conditioning provides an additional benefit with RAI, while the force-disabled comparison in §IV-B1 shows that the benefit of recursive collection persists without force observations.

V. Discussion

The matched-episode comparisons show that RAI substantially improves reliability over pure teleoperation, with further performance gains from force conditioning for contact-rich tasks. Our experiments evaluate accelerated execution and human correction as the combined RAI collection process, and has not yet isolated every component independently. Since the experimental time and real-robot cost limited the full combinatorial ablation, we prioritised matched-budget comparisons and the force ablation study in this paper. Evaluation is further limited to one robot platform and the focused tasks. Also higher playback scales are constrained by physical joint-velocity limits and inference latency, while broader generalisation and total human effort remain for future evaluation.

VI. Conclusion

We introduced RAI, Recursive Accelerated Imitation, which trains successive robot policies from recorded execution with human corrections. RAI starts from teleoperation, accelerates

the current policy during collection, and aggregates the resulting episodes for the next supervised imitation update. At the same episode budget and with the same policy architecture, recursively training on the robot's own accelerated, human-corrected closed-loop experience produces substantially higher reliability than collecting the same number of episodes through teleoperation alone. For example, the final RAI policy achieves 92.3% mean full-trial success across four manipulation tasks, compared with 62.5% for the same architecture trained on pure teleoperation. Moreover, when it comes to contact-rich tasks, force conditioning further augments performance for handling force-sensitive tasks towards higher success rate.

REFERENCES

[1] T. Z. Zhao *et al.*, "Learning fine-grained bimanual manipulation with low-cost hardware," *arXiv preprint arXiv:2304.13705*, 2023.

[2] C. Chi *et al.*, "Diffusion policy: Visuomotor policy learning via action diffusion," *arXiv preprint arXiv:2303.04137*, 2023.

[3] K. Black *et al.*, "π_0: A vision-language-action flow model for general robot control," in *Proceedings of Robotics: Science and Systems (RSS)*, 2025.

[4] B. ju Kim *et al.*, "Espada: Execution speedup via semantics aware demonstration data downsampling for imitation learning," *arXiv preprint arXiv:2512.07371*, 2025.

[5] K. Black *et al.*, "Real-time execution of action chunking flow policies," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[6] A. Brohan *et al.*, "Rt-1: Robotics transformer for real-world control at scale," *arXiv preprint arXiv:2212.06817*, 2022.

[7] O. X.-E. Collaboration *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models," *arXiv preprint arXiv:2310.08864*, 2023.

[8] M. J. Kim *et al.*, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[9] NVIDIA *et al.*, "Gr00t n1: An open foundation model for generalist humanoid robots," *arXiv preprint arXiv:2503.14734*, 2025.

[10] Y. Liu *et al.*, "Bidirectional decoding: Improving action chunking via guided test-time sampling," in *International Conference on Learning Representations (ICLR)*, 2025.

[11] D. Jing *et al.*, "Mixture of horizons in action chunking," *arXiv preprint arXiv:2511.19433*, 2025.

[12] A. Prasad *et al.*, "Consistency policy: Accelerated visuomotor policies via consistency distillation," *arXiv preprint arXiv:2405.07503*, 2024.

[13] S. H. Høeg *et al.*, "Streaming diffusion policy: Fast policy synthesis with variable noise diffusion models," *arXiv preprint arXiv:2406.04806*, 2024.

[14] Z. Li *et al.*, "Flashvla: Streaming action decoding for fast and asynchronous vla inference," *arXiv preprint arXiv:2608.27384*, 2026.

[15] N. R. Arachchige *et al.*, "Sail: Faster-than-demonstration execution of imitation learning policies," in *Conference on Robot Learning (CoRL)*, 2025.

[16] Q. Hu *et al.*, "Autospeed: Annotation-free stage-adaptive motion speed learning for robot manipulation," in *European Conference on Computer Vision (ECCV)*, 2026.

[17] D. D. Yuan *et al.*, "Speedtuning: Speeding up policy execution with lightweight reinforcement learning," *arXiv preprint arXiv:2608.09138*, 2026.

[18] H. Park *et al.*, "Proleptic temporal ensemble for improving the speed of robot tasks generated by imitation learning," *Journal of Korea Robotics Society*, vol. 20, no. 1, pp. 61–68, 2025.

[19] L. Guo *et al.*, "Demospeedup: Accelerating visuomotor policies via entropy-guided demonstration acceleration," in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 305, 2025, pp. 599–609.

[20] R. R. Sanchez *et al.*, "Volt: Vision and language trajectory segmentation for faster-than-demonstration policies," *arXiv preprint arXiv:2606.06323*, 2026.

[21] D. Jing *et al.*, "Tempovla: Learning speed-controllable vision-language-action policies," *arXiv preprint arXiv:2606.06491*, 2026.

[22] T. Nam *et al.*, "Speedaug: Policy acceleration via tempo-enriched policy and rl fine-tuning," *arXiv preprint arXiv:2512.00062*, 2025.

[23] S. Ross *et al.*, "A reduction of imitation learning and structured prediction to no-regret online learning," in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[24] M. Kelly *et al.*, "Hg-dagger: Interactive imitation learning with human experts," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 8077–8083.

[25] H. Liu *et al.*, "Robot learning on the job: Human-in-the-loop autonomy and learning during deployment," in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[26] J. Luo *et al.*, "Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning," *Science Robotics*, vol. 10, no. 105, p. eads5033, 2025.

[27] P. Intelligence *et al.*, "$\pi^*_{0.6}$: a vla that learns from experience," *arXiv preprint arXiv:2511.14759*, 2025.

[28] K. Yamane *et al.*, "Refinement of accelerated demonstrations via incremental iterative reference learning control for fast contact-rich imitation learning," *arXiv preprint arXiv:2604.16850*, 2026.

[29] Z. Li *et al.*, "Hand-in-the-loop: Improving vla policies for dexterous manipulation via seamless hand-arm intervention," *arXiv preprint arXiv:2605.15157*, 2026.

[30] Y. Han *et al.*, "Dexhil: A human-in-the-loop framework for vision-language-action model post-training in dexterous manipulation," *arXiv preprint arXiv:2603.09121*, 2026.

[31] R. Zurbrügg *et al.*, "Vr-dagger: Immersive vr for dexterous data collection and uncertainty-guided on-policy correction," *arXiv preprint arXiv:2605.27114*, 2026.

[32] K. Bousmalis *et al.*, "Robocat: A self-improving generalist agent for robotic manipulation," *Transactions on Machine Learning Research*, 2023.

[33] J. Luo *et al.*, "Serl: A software suite for sample-efficient robotic reinforcement learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[34] W. Xiao *et al.*, "Self-improving vision-language-action models with data generation via residual rl," in *International Conference on Learning Representations (ICLR)*, 2026.

[35] J. A. Collins *et al.*, "Forcesight: Text-guided mobile manipulation with visual-force goals," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[36] H. Xue *et al.*, "Reactive diffusion policy: Slow-fast visual-tactile policy learning for contact-rich manipulation," in *Proceedings of Robotics: Science and Systems (RSS)*, 2025.

[37] J. Yu *et al.*, "Forcevla: Enhancing vla models with a force-aware moe for contact-rich manipulation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[38] J. Huang *et al.*, "Tactile-vla: Unlocking vision-language-action model's physical knowledge for tactile generalization," *arXiv preprint arXiv:2507.09160*, 2025.

[39] K. H. Wong *et al.*, "Beyond implicit force: Evaluating explicit force-torque proxies in action chunking with transformers," *arXiv preprint arXiv:2607.14578*, 2026.

[40] D. Lim *et al.*, "Mob-net: Limb-modularized uncertainty torque learning of humanoids for sensorless external torque estimation," *The International Journal of Robotics Research*, vol. 44, no. 1, pp. 96–128, 2025.

[41] D. Lim *et al.*, "Proprioceptive external torque learning for floating base robot and its applications to humanoid locomotion," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.

[42] J. J. Liu *et al.*, "Factr: Force-attending curriculum training for contact-rich policy learning," in *Proceedings of Robotics: Science and Systems (RSS)*, 2025.

[43] C. Cuan *et al.*, "Leveraging haptic feedback to improve data quality and quantity for deep imitation learning models," *IEEE Transactions on Haptics*, vol. 17, no. 4, pp. 984–991, 2024.

[44] S. Oh *et al.*, "Factr 2: Learning external force sensing for commodity robot arms improves policy learning," *arXiv preprint arXiv:2606.12406*, 2026.

[45] H. Zhang *et al.*, "Hy-embodied-0.5-vla: From vision-language-action models to a real-world robot learning stack," *arXiv preprint arXiv:2606.14409*, 2026.